6

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks **COURSE CODE: CSA071**

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Code of Conduct:

I Abhilash (192525462.) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

36

QUESTIONS:4 **Total Marks:40**

1.A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2.A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

aQuestions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification 2	Logical solutions with adequate justification 2	Basic solutions with weak justification 1	Incorrect or no solution provided 1
Clarity	10	Clear, well-structured, and easy to understand 2	Organized and understandable presentation 2	Limited clarity and structure 1	Poor clarity; difficult to understand 1

16 16 8 8

36

Case Study 1 — Video Conferencing Quality Degradation

Scenario

A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

1. Is the Problem Primarily Transport-Layer or Application-Layer?

The symptoms — voice distortion, frozen frames, delayed communication — combined with the measured 8% packet loss and 180 ms jitter, point mainly to the transport layer, specifically to how the underlying protocol (or a poor protocol choice) handles loss and timing for real-time media. The evidence for this diagnosis:

- Jitter (180 ms) is a transport/network-level metric — it measures variation in inter-packet arrival time caused by queuing and route changes, not something the application layer controls directly.
- Packet loss (8%) is extremely high for real-time media; ITU-T guidance treats even 1% loss as noticeable and >5% as severely degrading for voice/video. This magnitude of loss almost always originates below the application — congested links, insufficient QoS prioritization, or a transport protocol reacting badly to loss.
- If the platform is (incorrectly) using a TCP-based transport for the live audio/video streams, TCP's reliable-delivery guarantee would force retransmission and in-order delivery of media packets that are already late — this produces exactly the 'frozen frame, catch-up' behaviour described, because TCP will stall newer frames behind a retransmission of an old, now-useless one.

That said, application-level buffering strategy is a secondary contributing factor, not the root cause: if the jitter buffer is too small, 180 ms of jitter will cause audible dropouts even over a well-behaved transport; if it is too large, it adds delay on top of network delay. The recommendation below therefore treats this as primarily a transport-layer problem with an application-layer buffering component that must be tuned together.

2. Justification Based on Real-Time Multimedia Requirements

Real-time multimedia has fundamentally different delivery requirements from typical data transfer, which is why the transport choice matters so much here:

Requirement	File/Web Transfer (TCP fits)	Live Audio/Video (needs UDP-style transport)
Timeliness	Not critical — data can arrive late	Critical — a late packet is worse than a lost one

Ordering	Must be strict, in-order	Loose — a few out-of-order/lost frames are tolerable
Retransmission	Required for correctness	Counter-productive — retransmitted data usually arrives too late to render
Acceptable loss	Effectively zero	Small, bounded loss (a few %) is acceptable and concealable
Latency budget	Seconds are fine	ITU-T G.114 recommends < 150 ms one-way for acceptable voice quality

Because TCP enforces reliable, in-order delivery, any loss forces it to pause and retransmit, and its congestion-control algorithm reduces the sending rate after loss — both are exactly wrong for real-time media, where a stale audio/video frame is worthless and should be dropped and concealed, not resent. This is why real-time conferencing protocols are built on UDP (carrying RTP/RTCP), which delivers packets as soon as they arrive, tolerates loss, and leaves recovery decisions (skip, conceal, request a key-frame) to the application, which can make a timing-aware choice that TCP cannot.

3. Recommended Improvements

Protocol-Level

- Confirm the platform uses UDP-based RTP/SRTP for media (not TCP or a TCP-tunnelled fallback); if the client is falling back to TCP (common when UDP is blocked by a firewall), open/allow the required UDP port ranges.
- Enable Forward Error Correction (FEC) and/or NACK-based selective retransmission at the RTP layer so isolated losses are concealed or repaired without stalling the whole stream, unlike blanket TCP retransmission.
- Deploy DSCP/QoS marking (e.g. EF for voice, AF41 for video) on the network so conferencing traffic is prioritized over bulk/background traffic during peak hours, directly reducing queuing-induced jitter and loss.
- Use adaptive bitrate / congestion control designed for real-time media (e.g. Google Congestion Control used by WebRTC) so the sender backs off bandwidth smoothly instead of the video simply freezing.

Application-Level

- Tune the receiver's adaptive jitter buffer to track the measured 180 ms jitter rather than using a fixed buffer — too small causes underruns/glitches, too large adds unnecessary delay.
- Enable packet-loss concealment (PLC) for audio and frame-skipping/error-concealment for video so a lost packet degrades quality gracefully instead of freezing the call.
- Route through the nearest regional media relay/SFU (Selective Forwarding Unit) to shorten the path and reduce the chance of congestion-induced loss during peak hours.
- Monitor and auto-switch codecs/resolution (simulcast) based on real-time loss/jitter telemetry so quality degrades in a controlled way rather than the session breaking down.

Case Study 2 — High DNS Resolution Time (Global Web Services)

> **Scenario**
> *A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.*

1. Reasons Behind the High DNS Resolution Time

- Geographic distance to authoritative servers: if the authoritative DNS servers are centralized in one region, employees on other continents incur a long round-trip time purely from physical distance/propagation delay before they even reach the name server.
- Deep referral chain / recursive lookup depth: a full recursive resolution (root → TLD → authoritative) with no caching in between forces multiple sequential round trips, each adding latency, especially if any hop is far away.
- Low or misconfigured TTL values: if DNS records have very short Time-To-Live values, resolvers are forced to repeat the full lookup far more often instead of serving a cached answer, multiplying the number of 800 ms lookups users experience.
- Single point of resolution (no Anycast): if all queries funnel to one primary server/location rather than being routed to the topologically nearest instance, every query pays the full distance penalty regardless of where the user is.
- Resolver-side congestion or overload: a single DNS server handling global query volume can queue requests during peak hours, adding processing delay on top of network delay.
- Lack of client-side/browser DNS pre-fetching: without pre-fetching, the DNS lookup sits directly in the critical path of the very first request, so its full cost is felt before the homepage can even begin loading.

2. Proposed DNS Architecture (Target: < 50 ms Lookup)

Technique	How It Reduces Lookup Time
Anycast DNS	The same DNS IP address is announced from multiple geographically distributed points-of-presence; BGP routes each user's query to the nearest instance automatically, cutting propagation delay from hundreds of ms to typically single-digit/low-double-digit ms.
DNS Pre-fetching	The browser/OS resolves domains it predicts will be needed (e.g. links on the current page, known internal services) in the background before the

	user actually requests them, so the 'visible' lookup time on the critical path drops to near 0 ms for pre-fetched names.
TTL Optimization	Setting an appropriately longer TTL (e.g. 300–3600 s for stable records) lets resolvers and clients reuse cached answers far more often, so only a small fraction of requests pay the full resolution cost; TTL is kept short only for records that must change quickly (e.g. failover records).
Geo-distributed secondary/authoritative servers	Placing authoritative name servers in each major region (in addition to Anycast) ensures that even a cache miss resolves against a nearby server rather than crossing continents.
Local caching resolvers (e.g. on-premises or per-region recursive resolvers)	Corporate offices/regions run their own caching resolver, so repeated lookups for the same popular internal/external domains are served from local memory almost instantly.

Combined, this architecture works as follows: a user's query first hits a local caching resolver (near-instant if cached); on a cache miss, Anycast routes the query to the nearest of several geographically distributed authoritative servers; pre-fetching ensures many lookups happen off the critical path entirely; and sensible TTLs keep the cache-hit rate high so the 800 ms 'cold' lookup path is rarely exercised. Because Anycast additionally provides multiple redundant instances of the same service address, continuous availability during a server failure is achieved automatically: BGP simply withdraws the route to the failed instance and traffic converges on the next-nearest healthy one, without any client-visible change of IP address.

3. Advantages and Limitations of the Proposed Architecture

Aspect	Advantages	Limitations
Performance	Anycast + regional resolvers cut round-trip distance dramatically; pre-fetching removes lookups from the critical path entirely	Anycast performance depends on BGP routing quality; poorly peered ISPs may still route to a farther PoP
Scalability	Additional PoPs can be added without changing the service's public IP address; load is naturally distributed by proximity	Managing many geographically distributed authoritative servers increases operational and synchronization complexity
Availability / Fault tolerance	Anycast automatically reroutes around a failed PoP via BGP convergence with no client reconfiguration; multiple authoritative servers avoid a single point of failure	BGP convergence is not instantaneous — a brief window of packet loss/timeouts can occur during failover; misconfigured Anycast can cause asymmetric routing issues
Consistency / Freshness	Longer TTLs reduce load and latency	Longer TTLs mean DNS-based failover or record changes propagate more slowly, so TTL must be tuned per record type (long for stable records, short for failover-critical ones)

Cost / Complexity	Significant, measurable UX improvement for a global user base	Higher infrastructure cost (multiple PoPs, monitoring, BGP management) and requires DNS/network engineering expertise to operate correctly

Case Study 3 — Centralized DNS Bottleneck at Global Scale

Scenario

A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

1. Impact of Centralized DNS Architecture on Application Performance

- Single point of congestion: every user worldwide funnels through one server, so during high-traffic events (promotions) the server's query queue grows, adding processing/queuing delay on top of network latency — directly explaining the 800 ms figure.
- Distance penalty for all remote users: users far from the single server's location always pay the full propagation delay; there is no 'nearest instance' to shorten the path, unlike a distributed design.
- Single point of failure: if the primary server becomes overloaded or fails, DNS resolution for the entire global user base is affected simultaneously — there is no automatic failover path, risking a total outage rather than a localized slowdown.
- Poor scalability under load spikes: promotional events cause sudden, large increases in concurrent users; a single server has a hard capacity ceiling, so performance degrades sharply exactly when it matters most (peak business value).
- Cascading effect on application performance: since DNS resolution sits before any application request can even begin, an 800 ms DNS delay is added in full to every page load / API call, amplifying perceived slowness across the whole platform.

2. Distributed DNS Solution Design

A distributed architecture should combine the following layers:

1. Anycast-based authoritative DNS: deploy the authoritative name servers behind a single Anycast IP announced from multiple points-of-presence (PoPs) across continents, so each query is BGP-routed to the nearest healthy instance rather than a fixed single server.

2. Geographically distributed secondary servers: maintain multiple synchronized authoritative servers (primary + secondaries) in different regions, so no single server is a bottleneck or single point of failure; use standard zone-transfer/replication to keep them consistent.
3. Multi-tier caching: rely on recursive resolvers at the ISP/regional level and client-side OS/browser caching, backed by well-tuned TTLs, so the large majority of repeat queries never reach the authoritative layer at all.
4. Health-checked load balancing / DNS failover: integrate active health checks so that if a PoP or backend service becomes unhealthy, its records are automatically withdrawn or redirected (e.g. via short-TTL failover records) to a healthy region.
5. Elastic capacity for peak events: provision the distributed nodes to auto-scale (or over-provision ahead of known promotional events) so query-handling capacity grows with demand instead of queuing at a fixed-capacity single server.

With this design, a user's query is answered by the nearest of several redundant, synchronized servers; if any one instance fails or is overloaded, Anycast/BGP convergence and health checks redirect traffic to the next-nearest healthy instance automatically, without requiring any change on the client side — turning the earlier single point of failure into a resilient, horizontally scalable service.

3. Influence of TTL, Anycast Routing, and DNS Caching

Factor	Effect on Performance	Effect on Fault Tolerance
TTL Values	Longer TTL → higher cache-hit rate → far fewer full lookups → lower average resolution time and lower load on authoritative servers	Longer TTL → slower propagation of failover/record changes, so critical failover records should use short TTLs even though this slightly increases their query load
Anycast Routing	Routes each query to the topologically nearest PoP, minimizing propagation delay regardless of where in the world the user is	Automatically reroutes traffic away from a failed or unreachable PoP once BGP converges, without any change to the service's public IP or client configuration
DNS Caching (multi-tier: OS, browser, ISP resolver)	Absorbs the vast majority of repeat queries locally, so most user-visible lookups are effectively 0 ms after the first resolution	Reduces load reaching the authoritative tier, indirectly improving resilience by keeping the authoritative servers from being overwhelmed even if one region experiences a traffic spike

Together, these three factors form a layered defence: caching minimizes how often the authoritative layer is even contacted; Anycast minimizes the delay and maximizes resilience when it is contacted; and correctly tuned TTLs balance the trade-off between caching efficiency (favouring long TTLs) and rapid failover response (favouring short TTLs) for the specific records that need it.

Case Study 4 — Latency Spike in an Electronic Stock Trading Platform

Scenario

A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

1. Three TCP-Related Factors Behind the Latency Increase

a) Flow Control (Receive Window Exhaustion)

TCP flow control uses the receiver's advertised window to prevent a fast sender from overwhelming a slow receiver's buffer. At market open, order volume spikes sharply; if the receiving application (matching engine or gateway) cannot drain its socket buffer as fast as orders arrive, its advertised window shrinks toward zero. The sender then must pause and wait (a 'zero-window' condition) before sending more data, directly adding delay to order submission and acknowledgment — exactly the 'excessive buffering' observed.

b) Nagle's Algorithm (Send-Side Coalescing Delay)

Nagle's algorithm delays sending small segments in order to coalesce them into larger ones, to improve efficiency on the sender side. Trading messages (order requests, acknowledgments) are typically small and latency-critical. If Nagle's algorithm is enabled, the stack may hold a small outgoing message waiting either for more data to batch with it or for an ACK of previously sent data (interacting badly with Delayed ACK on the receiver side) — this classic 'Nagle + Delayed ACK' interaction can by itself introduce tens of milliseconds of added latency per message, which lines up closely with the observed rise from 1 ms to 40 ms.

c) SYN Queue Limitations (Connection Establishment Backlog)

Each new TCP connection begins with a SYN arriving at the listening socket's SYN queue (backlog) before the three-way handshake completes and the connection is accepted by the application. At market open, many trading clients/sessions attempt to (re)connect simultaneously; if the configured SYN backlog (and/or the accept queue) is too small, incoming SYNs are queued, delayed, or dropped and must be retried, lengthening connection establishment time — consistent with the engineers' observation of 'longer connection establishment times' and contributing to the retransmissions seen (retransmitted SYNs when the queue overflows).

2. How Each Factor Affects High-Frequency Transaction Processing

Factor	Effect on Transaction Processing
Flow control (window exhaustion)	Order/acknowledgment messages queue up waiting for buffer space instead of being transmitted immediately, directly delaying trade

	confirmations during the highest-value trading window (market open) and risking order timeouts.
Nagle's algorithm	Small, latency-critical order messages are artificially delayed by the stack itself even when the network has spare capacity, adding tens of milliseconds of pure software-induced latency per round trip — unacceptable for high-frequency trading, where microsecond/low-millisecond response times are the norm.
SYN queue limitations	New or reconnecting trading sessions experience delayed or failed connection setup during the peak connection-attempt surge at market open, pushing back the point at which a client can begin submitting orders at all, and increasing retransmitted SYNs which further congests the link.

3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm on trading connections by setting TCP_NODELAY on the sockets used for order submission/acknowledgment, so small latency-critical messages are sent immediately instead of being batched.
- Disable or tune Delayed ACK (e.g. TCP_QUICKACK on Linux) on the peer side so acknowledgments are not held back and interacting with a disabled Nagle's algorithm to compound delay.
- Increase the receive/send socket buffer sizes (SO_RCVBUF / SO_SNDBUF) and enable TCP window scaling so the effective window at market-open traffic volumes does not collapse to zero, keeping data flowing under flow control.
- Increase the SYN backlog and accept queue sizes (e.g. Linux net.core.somaxconn and the listen() backlog parameter, plus net.ipv4.tcp_max_syn_backlog) so a surge of simultaneous connection attempts at market open can be queued and accepted without being dropped or retried.
- Enable SYN cookies as a safety net for handling connection-request bursts without exhausting the SYN queue, while still allowing legitimate high-volume reconnections to succeed.
- Pre-establish and keep-alive persistent TCP connections (connection pooling) for known trading clients ahead of market open, so the handshake cost is paid once, off the critical path, rather than repeatedly during the highest-traffic window.
- Consider a less conservative initial congestion window / a congestion-control algorithm suited to low-latency bursts (e.g. BBR) to reduce ramp-up delay for short-lived, latency-sensitive trading connections.
- Monitor and tune these parameters specifically for the market-open traffic profile (short burst of very high connection and message rate), validating improvements with the same retransmission/latency metrics that revealed the problem.